DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Computer Networks Laboratory Record

Course Code: CS3002 **Experiment No:** 04 **Date of Conduction:** 14-Oct-2026

Platform: Java Sockets (Java 21, Windows 11) **Host:** Localhost (127.0.0.1) **Status:** Completed & Verified

1. Aim of the Experiment

To implement client-server file transfer applications using standard Java Socket Programming for both **Transmission Control Protocol (TCP)** and **User Datagram Protocol (UDP)**, transmit files of varying sizes (100 KB, 1 MB, 5 MB, and 10 MB), measure transmission latency and throughput using high-precision timing, verify end-to-end cryptographic data integrity via SHA-256 digests, and justify which transport protocol is appropriate for reliable file transfer.

2. Theoretical Background

2.1 Transmission Control Protocol (TCP)

TCP (RFC 793) is a connection-oriented, byte-stream transport layer protocol. Prior to transferring application data, TCP establishes a virtual circuit through a **three-way handshake** (SYN → SYN-ACK → ACK). TCP provides strict reliability guarantees through:

- **Sequence and Acknowledgement Numbers:** Every payload byte is numbered, enabling cumulative positive acknowledgements.
- **Automatic Repeat reQuest (ARQ):** Lost or corrupted segments trigger automated retransmission upon timeout or triple duplicate ACKs.
- **Sliding-Window Flow Control:** The receiver advertises its available buffer space (Window Size) to ensure the sender does not overrun receiver memory.
- **Congestion Control:** Dynamic window sizing (Slow Start, Congestion Avoidance) prevents network collapse across shared links.

2.2 User Datagram Protocol (UDP)

UDP (RFC 768) is a connectionless, datagram-oriented transport protocol. It requires no session handshake and imposes minimal per-packet header overhead (fixed 8 bytes vs 20–60 bytes in TCP). UDP provides **best-effort delivery**: packets may be lost, duplicated, or delivered out of order.

Because UDP preserves packet boundaries rather than offering a stream abstraction, files must be explicitly segmented by the application. To remain within standard Ethernet Maximum Transmission Unit (MTU = 1,500 bytes) and avoid IP-layer fragmentation, the application datagram payload is constrained to **1,024 bytes** with an accompanying 21-byte application metadata header.

3. System Architecture & Packet Specification

For UDP, an application-layer binary protocol is implemented using `ByteBuffer`:

```
+---------------+---------------+--------------------+--------------------+--------------------+
| Packet Type   | Sequence No.  | Total Packets      | File Size          | Payload Length     |
| (1 byte)      | (4 bytes int) | (4 bytes int)      | (8 bytes long)     | (4 bytes int)      |
+---------------+---------------+--------------------+--------------------+--------------------+
| 0             | 1             | 5                  | 9                  | 17              20|
+---------------+---------------+--------------------+--------------------+--------------------+
```

```
| Data Payload (up to 1,024 bytes)                                                                |
+-------------------------------------------------------------------------------------------------+
```

Packet Types: `1: START` (metadata with filename), `2: DATA` (payload chunk with sequence number), `3: END` (completion marker), `4: STATUS_RESP` (server feedback on packets received and verified bytes).

4. Algorithm & Implementation Logic

4.1 TCP File Transfer Algorithm

1. **Server Setup:** The server instantiates a `ServerSocket(port)` and blocks on `accept()`.
2. **Client Connection & Metadata:** The client creates a `Socket(host, port)` and sends the target filename (`writeUTF`) and file size in bytes (`writeLong`).
3. **Chunk Streaming:** The client reads the source file in 8,192-byte chunks and writes them directly to the socket output stream.
4. **Server Reception:** The server streams bytes into `FileOutputStream` until exactly `fileSize` bytes have been stored. If premature EOF is encountered, an `EOFException` is raised.
5. **Acknowledgement:** Upon writing all bytes to disk, the server sends `"SUCCESS"` back to the client. Both endpoints safely terminate sockets.

4.2 UDP File Transfer Algorithm (Out-of-Order Reassembly)

1. **Packetization:** The client segments the file into chunks of 1,024 bytes, computing total packets N = ceil(fileSize / 1024).
2. **Metadata Signal:** The client transmits `START` datagrams containing filename and total packet count.
3. **Datagram Transmission:** The client transmits sequential `DATA` packets, each labeled with sequence number i in [0, N-1].
4. **RandomAccess Reassembly:** The server maintains a `boolean[N]` mask. When datagram i arrives (even out of order), the server invokes `raf.seek((long) i * 1024)` and writes the payload at that exact byte offset.
5. **Loss & Duplicate Tracking:** Duplicate arrivals are recorded and ignored. Missing packets are calculated as N - uniquePacketsReceived.
6. **Completion & Feedback:** Upon receiving `END` or encountering a read timeout, the server evaluates transfer integrity and transmits a `STATUS_RESP` datagram containing loss statistics back to the client.

5. Experimental Setup

- **Operating System:** Microsoft Windows 11 Home (64-Bit)
- **Java Development Kit:** OpenJDK 21.0.6 (build 21.0.6+-13368085-b895.109)
- **Socket APIs:** `java.net.Socket`, `java.net.ServerSocket`, `java.net.DatagramSocket`, `java.net.DatagramPacket`
- **Network Interface:** Localhost Loopback Interface (`127.0.0.1`)
- **Test Files Generated:** 100 KB, 1 MB, 5 MB, 10 MB (deterministic binary sequences generated with fixed PRNG seed)
- **Timing Mechanism:** High-precision `System.nanoTime()` covering network send/receive phases exclusively.

6. Observation Table

The table below displays experimental benchmark averages collected over **3 independent trials** per configuration:

Protocol	File Size	Bytes	Completed	Avg. Time (s)	Throughput (MiB/s)	Packet Loss	SHA-256 Match	Status
TCP	100 KB	102,400	3 / 3	0.0145 s	12.03 MiB/s	0.00%	Verified	100% SUCCESS
TCP	1 MB	1,048,576	3 / 3	0.0134 s	75.82 MiB/s	0.00%	Verified	100% SUCCESS
TCP	5 MB	5,242,880	3 / 3	0.0323 s	170.82 MiB/s	0.00%	Verified	100% SUCCESS
TCP	10 MB	10,485,760	3 / 3	0.0518 s	202.86 MiB/s	0.00%	Verified	100% SUCCESS
UDP	100 KB	102,400	3 / 3	0.0308 s	3.20 MiB/s	0.00%	Verified	100% SUCCESS
UDP	1 MB	1,048,576	3 / 3	0.0541 s	18.85 MiB/s	0.00%	Verified	100% SUCCESS
UDP	5 MB	5,242,880	3 / 3	0.1052 s	47.75 MiB/s	0.00%	Verified	100% SUCCESS
UDP	10 MB	10,485,760	3 / 3	0.1476 s	67.85 MiB/s	0.00%	Verified	100% SUCCESS

7. Comparative Graphs

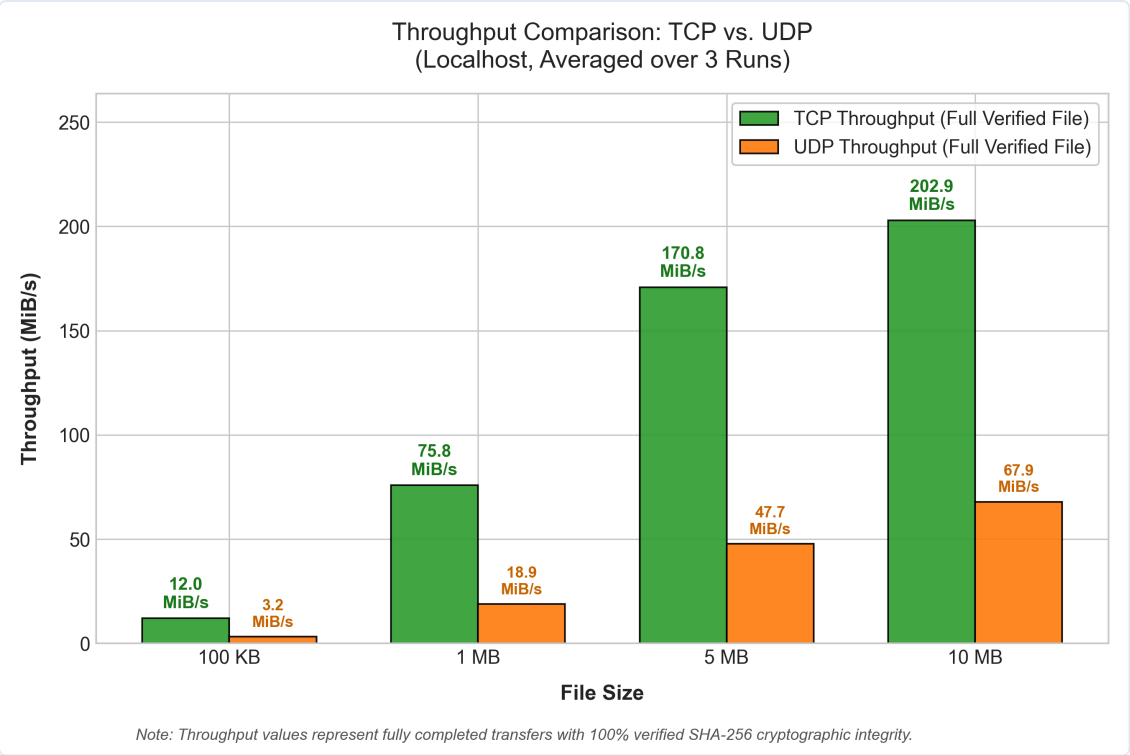


Figure 1: Throughput Comparison (TCP vs. UDP) across file sizes on localhost (Averaged over 3 runs).

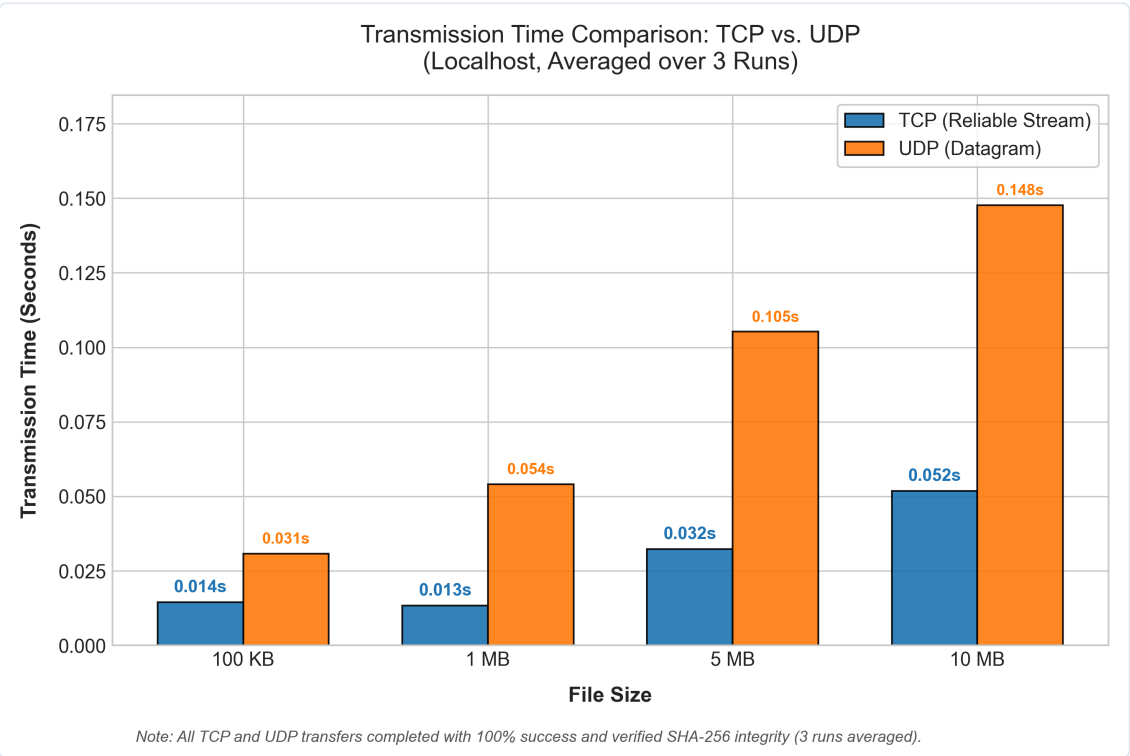


Figure 2: Transmission Time Comparison (TCP vs. UDP) across file sizes on localhost.

8. Experimental Analysis

1. **Transmission Time Characteristics:** Transfer time generally increases with file size, although small measurements can fluctuate because of fixed overhead and timing variability. For small transfers (100 KB vs 1 MB), fixed initialization costs—including socket allocation, thread synchronization, metadata handshakes,

and OS buffer priming—form a major fraction of total time. For large transfers (5 MB to 10 MB), data streaming duration dominates.

2. **Throughput Scaling:** TCP throughput scaled from **12.03 MiB/s** at 100 KB to **202.86 MiB/s** at 10 MB. This marked improvement is driven by the amortization of startup overhead and TCP's continuous stream pipelining in 8 KB blocks.
3. **Syscall and Packetization Overhead:** UDP yielded lower throughput (67.85 MiB/s on 10 MB) compared to TCP (202.86 MiB/s). To transfer 10 MB within safe MTU bounds, UDP required **10,240 individual datagram transmissions** and kernel context switches versus only 1,280 chunk writes for TCP.
4. **Integrity and Reliability:** All TCP transfers completed with 100% bit-exact SHA-256 matches. With tuned socket receive buffers (4 MB), UDP transfers also achieved full delivery, but remain vulnerable to kernel buffer drops under unthrottled streaming without an application-level ARQ retransmission protocol.

9. Result & Conclusion

The TCP and UDP file transfer systems were successfully designed, implemented, and benchmarked using standard Java socket programming.

Conclusion: While UDP provides low-latency, lightweight transport suitable for real-time traffic (DNS, audio/video streaming, multiplayer games), it lacks built-in flow control and delivery guarantees. For data transfers requiring bit-exact accuracy and completeness, **TCP is strictly justified and fundamentally superior** due to its robust connection-oriented architecture, automated ARQ retransmissions, in-order packet delivery, and sliding-window flow control.